Scraper API Service
Data Engineering Project

Prepared by:
Amran Al-gaafari

Bootcamp:

Data Engineering

Email:

amranalgaafari@gmail.com

Scraper API Service

Complete Project Documentation, Deployment Notes, and Study Guide

Project	Week-05 Scraper API Service
Bootcamp Repository	A1-Data-Engineering-Bootcamp
Project Folder	Week-05-Scraper-API-Service/Project-Scraper-API-Service
Prepared For	Data Engineering bootcamp assessment / project review
Purpose	Documentation, revision, and presentation reference

This document explains the project from architecture to commands, packaging, and deployment. It is intended to be used both as a submission guide and as a study/revision file.

1. Project Summary

This project implements a complete data engineering pipeline that starts with web scraping and ends with a background Linux service that can be installed using a Debian package.

The application scrapes laptop products from the Webscraper.io e-commerce test site, cleans and normalizes the records, stores them in PostgreSQL, exposes the data using FastAPI, runs as a systemd service, opens a network port through UFW, and can be packaged as a .deb file for easier deployment.

2. Why This Project Was Chosen

The chosen website provides commercial-style product pages with fields that fit a realistic business use case: product name, price, description, rating, product URL, image URL, and category. This makes the project more useful than a simple text example because it resembles a product catalog, a competitor price monitoring service, or a lightweight product ingestion pipeline.

PostgreSQL was used instead of SQLite because it is closer to production systems and aligns better with SQL topics covered in the bootcamp.

3. Source Website and Extracted Data

Source website

Base site: https://webscraper.io
Category page used: https://webscraper.io/test-sites/e-commerce/static/computers/laptops

Why this site

Public test site intended for scraping practice.
Stable HTML structure suitable for a bootcamp project.
Contains product-oriented data rather than generic text.
Allows explanation of data extraction, cleaning, storage, and serving.

Fields extracted from each product

Field	Example	Purpose
name	ThinkPad X240	Product identifier shown to users
price	$1311.99 → 1311.99	Commercial value used for sorting and analysis
description	12.5", Core i5-4300U, 8GB, 240GB SSD, Win7 Pro 64bit	Technical details of the product
rating	3	Customer/visual rating extracted from HTML
category	laptops	Logical grouping of the scraped page
product_url	/test-sites/e-commerce/static/product/35	Link to product detail page

Field	Example	Purpose
image_url	/images/test-sites/e-commerce/items/cart2.png	Visual asset reference

4. High-Level Architecture

The system is divided into layers so that each part has a clear responsibility.

Scraping layer: Fetches HTML from the source website and extracts raw product information using requests and BeautifulSoup.
Cleaning layer: Converts raw strings into normalized values, for example price text into numeric format and rating into integer format.
Storage layer: Stores the cleaned data in PostgreSQL using SQLAlchemy models and sessions.
API layer: Exposes the stored data through FastAPI endpoints for reading, filtering, searching, and triggering scraping.
Service layer: Runs the application in the background using systemd so that it starts automatically and restarts on failure.
Network layer: Allows access to the API through port 8000 using UFW firewall rules.
Packaging layer: Wraps the application into a Debian package so it can be installed more easily on another Ubuntu machine.

5. Project Structure

Main folders and files

```
project-scraper-api-service/
├── app/
│   ├── api.py
│   ├── cleaner.py
│   ├── config.py
│   ├── crud.py
│   ├── database.py
│   ├── models.py
│   ├── schemas.py
│   └── scraper.py
├── scripts/
│   ├── init_db.py
│   └── run_scraper.py
├── docs/
│   └── architecture.md
├── debian/
```

```
├── systemd/
│   └── scraper-api.service
├── data/
├── tests/
├── main.py
├── requirements.txt
├── README.md
└── .gitignore
```

Purpose of the main files

File / Folder	Role
app/scraper.py	Fetches and parses product data from the target website.
app/cleaner.py	Normalizes price, rating, and other fields before storage.
app/models.py	Defines the SQLAlchemy Product model.
app/database.py	Creates database engine, session, and base model.
app/crud.py	Contains database operations such as insert, search, filter, and truncate logic.
app/api.py	Defines FastAPI routes.
scripts/init_db.py	Creates tables in PostgreSQL.
scripts/run_scraper.py	Runs scraping manually from the command line.
systemd/scraper-api.service	Defines the Linux background service.
debian/	Contains files used for Debian packaging.

6. Detailed Data Flow

A request is sent to the target category page on Webscraper.io.
BeautifulSoup parses the HTML response.
For each product card, the scraper extracts the title, price, description, rating, category, product URL, and image URL.
The cleaning layer transforms the raw fields. Example: "$416.99" becomes 416.99, and the HTML rating attribute becomes an integer.
The cleaned records are converted into schema objects and inserted into PostgreSQL.
FastAPI reads the data from PostgreSQL and returns it through HTTP endpoints.
A systemd service keeps the API running in the background.
UFW allows incoming traffic on port 8000.
A Debian package can install the application, create the virtual environment, prepare PostgreSQL, and start the service.

7. Data Cleaning and Normalization

Cleaning tasks performed

Price strings were cleaned by removing the dollar symbol and converting the result to a numeric value.
Rating was extracted from the real HTML attribute data-rating instead of guessing from visual stars.
Product URLs and image URLs were built with safe URL joining so duplicated path segments were avoided.
Null checks were added to reduce failures if a page element is missing.

Examples

Raw Value	Cleaned Value	Reason
$416.99	416.99	Store as numeric data for sorting and filtering
data-rating="3"	3	Store rating as integer
Relative href	Absolute URL	Return stable links in the API

8. Database Design

Database used: PostgreSQL

PostgreSQL was chosen because it is closer to real production systems and was also part of the bootcamp topics. The project uses a single main table named products.

Products table schema

Column	Type	Nullable	Meaning
id	Integer / Serial	No	Primary key
name	String(255)	No	Product name
price	Numeric(10,2)	No	Product price

Column	Type	Nullable	Meaning
description	Text	Yes	Product technical details
rating	Integer	No	Rating value
category	String(100)	Yes	Category name
product_url	Text	Yes	Product page URL
image_url	Text	Yes	Image URL
created_at	Timestamp	Auto	Record insertion time

9. API Design and Endpoints

Main endpoints

Method + Path	Purpose	Example
GET /health	Health check for service availabilit y	curl http://127.0.0.1:8000/health
GET /products	Return all products	curl http://127.0.0.1:8000/products
GET /products/{product_id}	Return one product by ID	curl http://127.0.0.1:8000/products/3
GET /products/search?q=ThinkPa d	Search by product name	curl "http://127.0.0.1:8000/products/search?q=ThinkPa d"
GET /products/rating/3	Filter by rating	curl http://127.0.0.1:8000/products/rating/3
GET /products/cheap?limit=3	Return cheapest products	curl "http://127.0.0.1:8000/products/cheap?limit=3"
POST /products	Manually insert a product	curl -X POST ...
POST /scrape	Run scraping and refresh table contents	curl -X POST http://127.0.0.1:8000/scrape

10. Linux Service and Deployment

systemd service

The application was converted into a Linux service so that it can run in the background, be restarted automatically, and be enabled on system startup.

```
sudo systemctl start scraper-api.service

sudo systemctl status scraper-api.service

sudo systemctl enable scraper-api.service
```

Firewall (UFW)

Port 8000 was opened so the API can be reached through the network.

```
sudo ufw allow 8000/tcp

sudo ufw enable

sudo ufw status
```

11. Debian Package (Portable Version)

The project was packaged as a Debian package so that it can be installed on another Ubuntu machine without depending on the original personal project path.

Portable deployment decisions

Application files are installed under /opt/scraper-api-service.
The service runs using the fixed system path instead of /home/<user>/projects/...
The service runs under a system user named scraperapi.
The post-install script creates .venv, installs requirements, prepares PostgreSQL, initializes tables, opens port 8000, and starts the service.

Package build command

```
dpkg-buildpackage -us -uc -b
```

Package install command

```
sudo dpkg -i ../scraper-api-service_1.0.0-1_all.deb
```

12. Important Commands Used During the Project

```
wsl --status

wsl --set-default Ubuntu

sudo apt update && sudo apt upgrade -y

sudo apt install -y python3 python3-pip python3-venv git curl postgresql
postgresql-contrib ufw

python3 -m venv .venv

source .venv/bin/activate

pip install -r requirements.txt
```

```
python -m scripts.init_db

python -m scripts.run_scraper

uvicorn main:app --reload --host 0.0.0.0 --port 8000

psql -h localhost -U scraper_user -d scraper_db

sudo systemctl start scraper-api.service

sudo systemctl status scraper-api.service

sudo ufw allow 8000/tcp

dpkg-buildpackage -us -uc -b

sudo dpkg -i ../scraper-api-service_1.0.0-1_all.deb
```

13. Testing and Verification Checklist

Health endpoint returned {"status":"ok"}.
Products were visible through GET /products.
POST /scrape successfully refreshed the database.
Search endpoint returned ThinkPad products.
Rating filter endpoint returned only records with the requested rating.
Cheapest-products endpoint returned sorted results.
systemd showed scraper-api.service as active (running).
UFW status showed 8000/tcp as ALLOW.
The Debian package was built successfully and later installed successfully.
The installed package launched the service from /opt/scraper-api-service/.venv/bin/uvicorn.

14. Challenges and How They Were Solved

Challenge	Solution
WSL Ubuntu install was affected by regional network restrictions.	VPN / alternative install path was used and Ubuntu was installed successfully.
Running Python modules from the wrong path caused ModuleNotFoundError for app.	Scripts were run as modules from the project root, for example python -m scripts.init_db.
Database user had no permission on schema public.	Schema privileges and ownership were granted in the correct database.
HTML rating extraction initially returned 0 for all products.	The real HTML structure was inspected and rating was extracted from data-rating.
systemd service failed with Windows-style paths containing spaces.	Project was moved to a Linux path and later packaged into /opt/scraper-api-service.
Initial package depended on a personal path.	Package was redesigned to install to a fixed system path and create its own virtual environment.

15. Why This Design Was Chosen

A layered design makes the system easier to explain and maintain.
PostgreSQL reflects real SQL usage better than a file database.
FastAPI provides a clean API interface and automatic documentation.
systemd is the standard way to run persistent services on Ubuntu.
UFW is a simple and clear firewall tool for opening the API port.
Debian packaging improves repeatability and deployment portability.

16. Suggested Screenshots for Submission

curl http://127.0.0.1:8000/health
curl -X POST http://127.0.0.1:8000/scrape
curl http://127.0.0.1:8000/products
sudo systemctl status scraper-api.service
sudo ufw status
psql query showing the products table content
ls -lh .. showing scraper-api-service_1.0.0-1_all.deb

17. Conclusion

This project demonstrates a complete end-to-end engineering workflow: collection, transformation, storage, serving, deployment, and packaging. It started as a Python scraping script and ended as a reusable Ubuntu service packaged as a Debian installer. The final result matches the project requirement and also serves as a strong revision file for database, Linux, packaging, and API concepts.